LIGHTWEIGHT GAZE ESTIMATION FOR REAL-TIME MOBILE DEPLOYMENT VIA KNOWLEDGE DISTILLATION

YOUR NAME [YOURNAME@SEAS.UPENN.EDU]

ABSTRACT. We train small CNNs to regress 2D gaze (g_x, g_y) from face images, with an eye toward models that could run on a phone. The implementation swaps torchvision backbones for the teacher (ResNet-18, MobileNetV2, or MobileNetV3-Small) and gives several student options, including a hand-built depthwise network (`gaze_micro`, on the order of 50k parameters). Besides plain MSE on labels we try knowledge distillation: add an MSE term that pulls the student toward the teacher's output. On MPIIGaze we do not use every training frame each epoch; we cap at 10,000 samples per epoch with a fixed order flag in the dataloader, but we still evaluate on the full validation subjects. With the same number of epochs and similar hyperparameters, that cap actually gave us lower validation error than when we tried to chew through the entire training split first—we do not claim 10k always wins, only that it happened in our setup and probably has to do with overfitting and training time per epoch. ResNet-18 as a teacher still looked worse on val than the smaller nets. Numbers in the paper come from `evaluate.py` exports and the plotting scripts in the repo.

1. INTRODUCTION

Gaze from a webcam or front camera shows up in HCI demos, accessibility work, and driver monitoring. The accuracy vs. size tradeoff is annoying: ResNets are fine on a GPU, but a 50k-parameter student is closer to what you would actually ship.

This report describes a course codebase where the task is boring on purpose: resize the crop to 224×224, predict two numbers, train with Adam. The only twist we cared about was KD. We also wasted a few GPU hours discovering that a ResNet-18 teacher likes to memorize our MPII train split while validation flatlines, which is why most of the plots use MobileNet-sized teachers instead.

1.1. Contributions.

- PyTorch code: MPIIGaze reader or CSV, teacher/student switches (`gaze_micro` included), KD with weight α.
- Training logs to CSV, eval to JSON, and a `make_paper_figures.py` helper that writes PNGs we include below.
- A short empirical section: regression errors, parameter counts, and timing on one validation protocol.

2. BACKGROUND

Let $\mathbf{I} \in \mathbb{R}^{3\times H\times W}$ be the input image and $\mathbf{g} = (g_x, g_y)$ the label. The network outputs $\hat{\mathbf{g}} = f_\theta(\mathbf{I})$. Supervised loss:

$$\mathcal{L}_{\mathrm{gt}}(\theta) = \left\|\hat{\mathbf{g}} - \mathbf{g}\right\|_2^2.$$

Fix a teacher f_{θ_T}. Distillation adds

$$\mathcal{L}_{\mathrm{kd}}(\theta_S) = \left\|f_{\theta_S}(\mathbf{I}) - f_{\theta_T}(\mathbf{I})\right\|_2^2,$$

and we minimize $\mathcal{L}_{\mathrm{gt}}(\theta_S) + \alpha\mathcal{L}_{\mathrm{kd}}(\theta_S)$ with the teacher frozen. Classification KD uses temperature-soft targets; here we stayed with plain MSE on the 2-D vector because it was easy to debug.

3. RELATED WORK

MPIIGaze [1] is the dataset people cite for "in the wild" laptop gaze. MobileNetV3 [3] is one of the standard efficient backbones if you care about FLOPs. Hinton et al. [2] introduced distillation for classification; we borrowed the idea but applied it to regression.

4. APPROACH

Data. CSV mode expects `image_path`, `gaze_x`, `gaze_y`. The loader resizes to 224×224 and uses ImageNet mean/std because the torchvision weights assume that.

Models. Teacher: swap the classifier head for a linear layer with two outputs. Student: either another torchvision tiny model or `gaze_micro` (depthwise separable blocks, no ImageNet init).
Training. Adam, 10^{-4}, batch 64, usually 20 epochs unless we say otherwise. Best checkpoint = best validation score (MSE for teacher and baseline student; combined KD loss for the distilled run). On our machine we turned on AMP (`--amp`) for speed; CPU runs skip it.
Metrics. MSE, MAE, mean L2 between $\hat{\mathbf{g}}$ and $\mathbf{g}$, plus parameter count and ms/image on a dummy batch-1 forward after a short warmup (whatever `evaluate.py` prints).

5. Experimental Results

5.1. **Data and protocol.** We trained on MPIIGaze normalized patches via `--dataset mpiigaze`. Train and val use different people (e.g. held-out `p14`, `p15` in the README example). The important detail for reproducing our table: **train is subsampled to 10,000 frames per epoch** (`--mpi_max_train_samples 10000`), deterministic order. Val is **not** subsampled.

We log curves with `--metrics_csv`. After training, `evaluate.py --export_json` three times, then `build_eval_summary.py`, then `make_paper_figures.py` (outputs PNG at 300 dpi by default). The loss figure only uses the CSVs; the bar charts and size/speed figures read the merged JSON. If a KD run went 40 epochs, the plot script clips to the first 20 points by default so the x-axis is readable (`--max_plot_epochs`).

5.2. **Quantitative comparison.** Table 1 matches the merged eval JSON used for the bar charts. Figure 1 comes from the training CSVs.

Table 1. MPIIGaze validation split (`mpi_eval_split = val`), batch-1 latency on the same machine as training. Numbers from merged `evaluate.py` JSON (teacher/student checkpoints as in the repo README).

Model	MSE ↓	MAE ↓	Mean L2 ↓	Params	ms/img ↓
Teacher (MobileNetV3-Small)	0.00212	0.0261	0.0408	1 519 906	7.04
Student (`gaze_micro`, baseline)	0.00837	0.0521	0.0816	49 426	1.61
Student (`gaze_micro`) + KD	0.00641	0.0455	0.0713	49 426	1.56

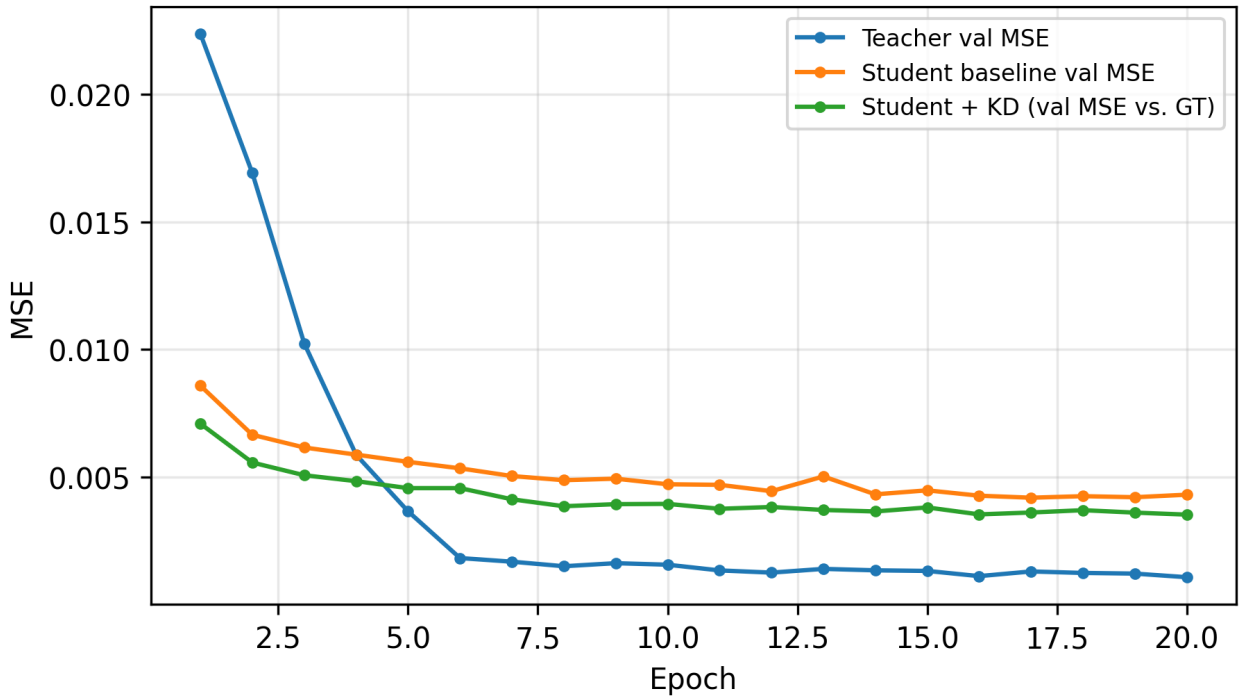

FIGURE 1. Validation MSE per epoch from `--metrics_csv`. KD line is student vs. ground truth on val (same as the other two curves). First 20 epochs shown by default if the log is longer.

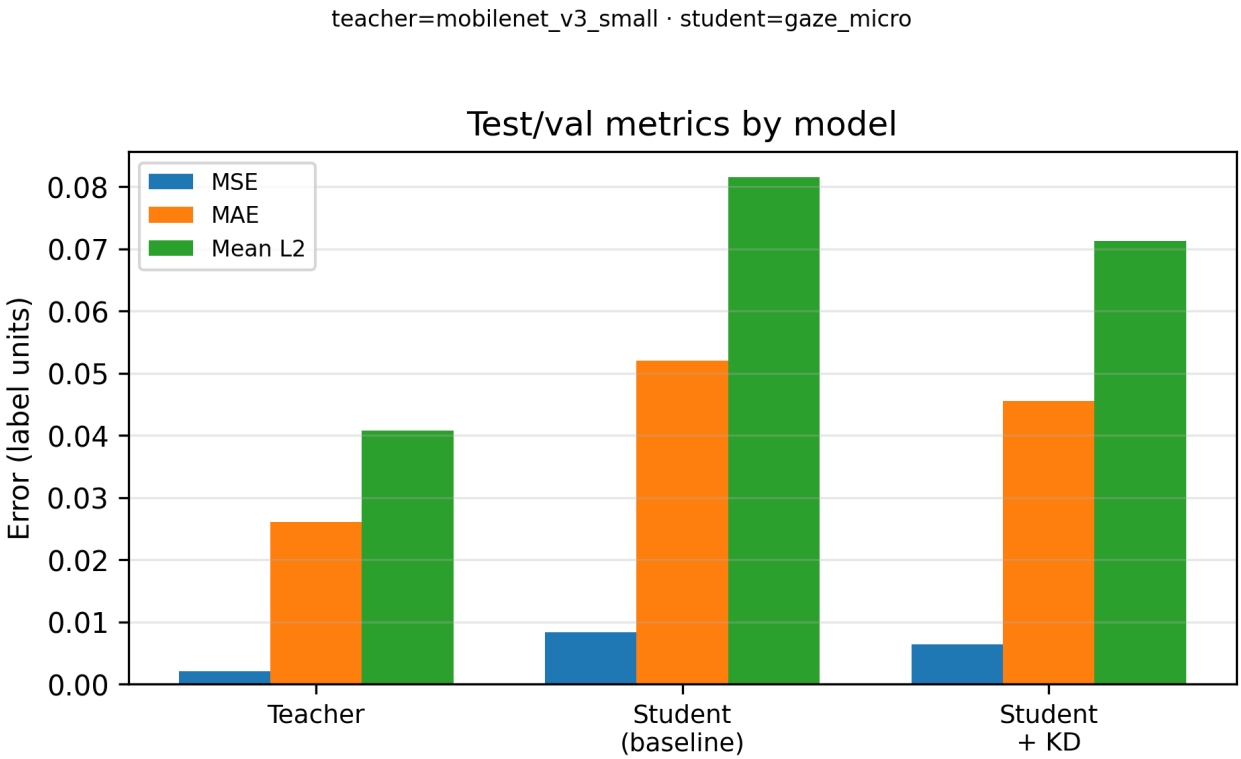

FIGURE 2. Bars from merged eval JSON. Should match Table 1.

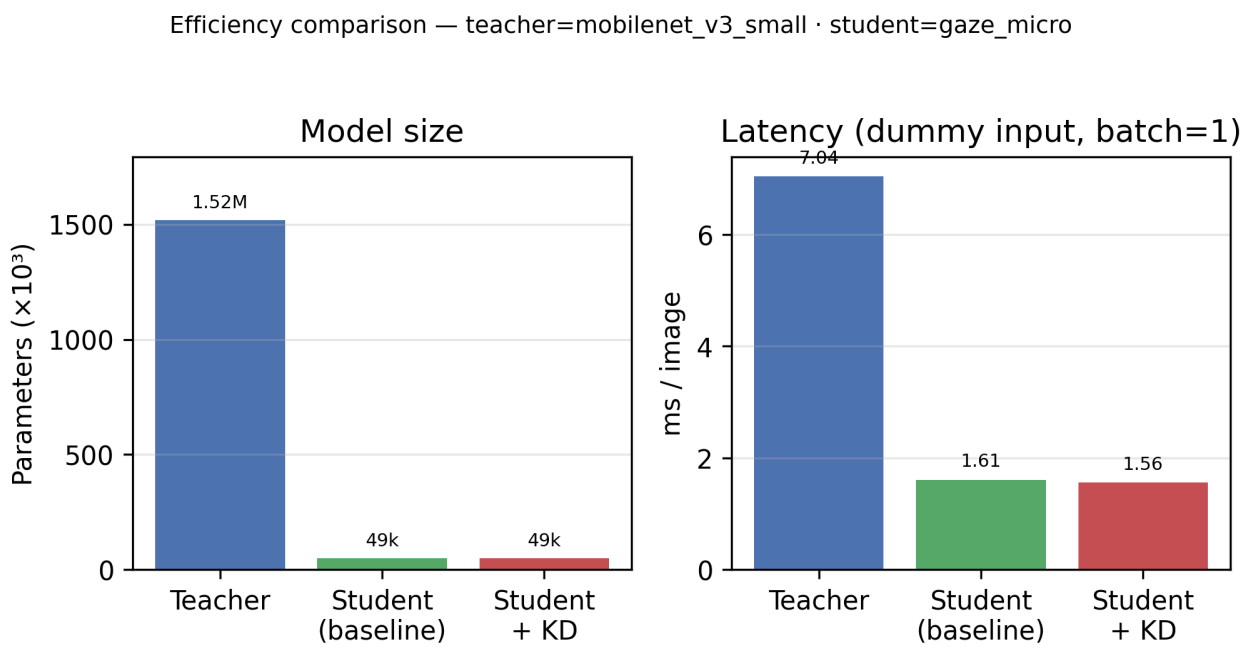

FIGURE 3. Left: parameters in thousands (annotations in M or k). Right: latency, batch 1, same setup as `evaluate.py`.

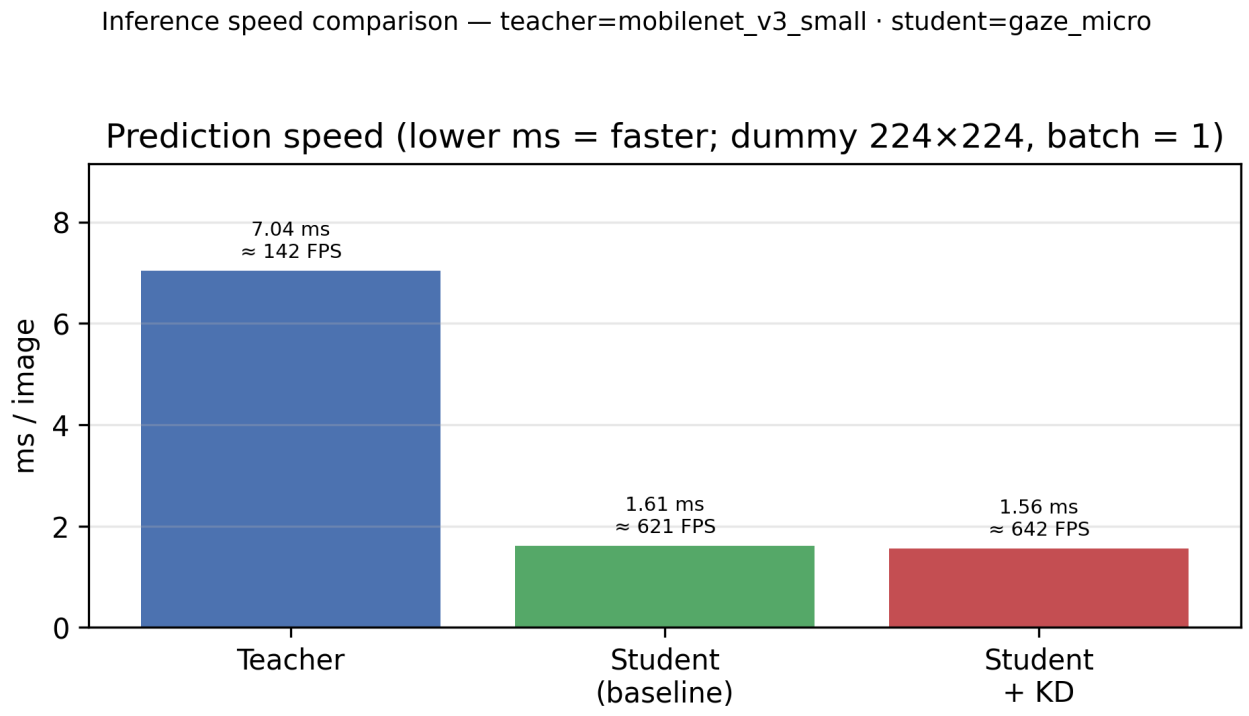

FIGURE 4. Same timing idea as the right panel of Figure 3, pulled out as its own figure with FPS text.

5.3. **Training on 10k examples per epoch.** Full MPIIGaze training is huge. One epoch over everything was slow enough that we started subsampling. After we fixed val to the full held-out people, we were surprised that 10k-per-epoch looked better on val than "eat the whole train set" with the same epoch count and roughly the same learning rate.

Smaller models (especially `gaze_micro`) have no business memorizing a million crops in 20 epochs anyway. Truncating each epoch might act like a crude regularizer. Shorter epochs also mean we actually finish updates in reasonable wall time, which matters when you are iterating on bugs. A third boring possibility: MPIIGaze is messy across sessions; a fixed 10k slice is not representative of the full distribution, so comparing to "full data" without retuning steps or LR is apples-to-oranges.

What we would try next if this were not a deadline: sweep N, train longer on full data with decay, and report angular error instead of (g_x, g_y) MSE.

5.4. **ResNet-18 vs. smaller teachers.** We tried ResNet-18 as a teacher because it is the default "serious" backbone. On our MPII setup it overfit: train MSE dropped, val did not follow. MobileNetV2 / MobileNetV3-Small behaved more sanely for the same (g_x, g_y) targets. So the "big teacher" story did not work out here; the patch-to-two-numbers mapping just is not ImageNet-classification hard.

5.5. **KD.** If the teacher is reasonable, KD sometimes shaves a bit off student val MSE relative to labels alone; sometimes it does not. α matters, and a bad teacher hurts. We mostly treated α as a knob to tune by hand.

6. Discussion

MSE in normalized label space is easy to implement but not what papers usually report for gaze (they want degrees). Getting angular error right needs geometry we did not implement. Augmentation (color jitter, etc.) might help but we kept the pipeline minimal so we could finish the assignment.

Exporting the student to ONNX or TFLite would be the next engineering step if this were a product. For the course, the PyTorch timings were enough.

The 10k cap is a reproducibility footnote: if someone reruns our commands, they need the same `--mpi_max_train_samples`, the same val persons, and the same arch flags or the numbers will drift.

References

References

[1] X. Zhang, Y. Sugano, M. Fritz, and A. Bulling, Appearance-based gaze estimation in the wild, in *Proc. IEEE CVPR*, 2015.
[2] G. Hinton, O. Vinyals, and J. Dean, Distilling the knowledge in a neural network, *arXiv:1503.02531*, 2015.
[3] A. Howard, M. Sandler, G. Chu, et al., Searching for MobileNetV3, in *Proc. IEEE ICCV*, 2019.